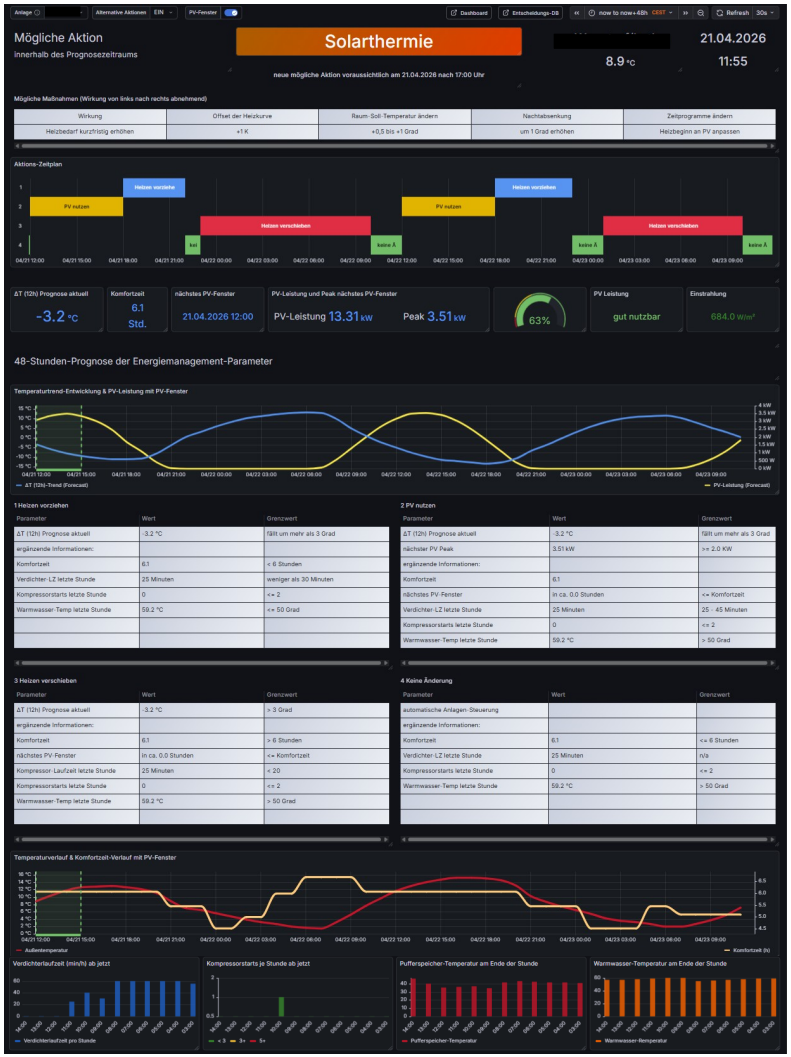
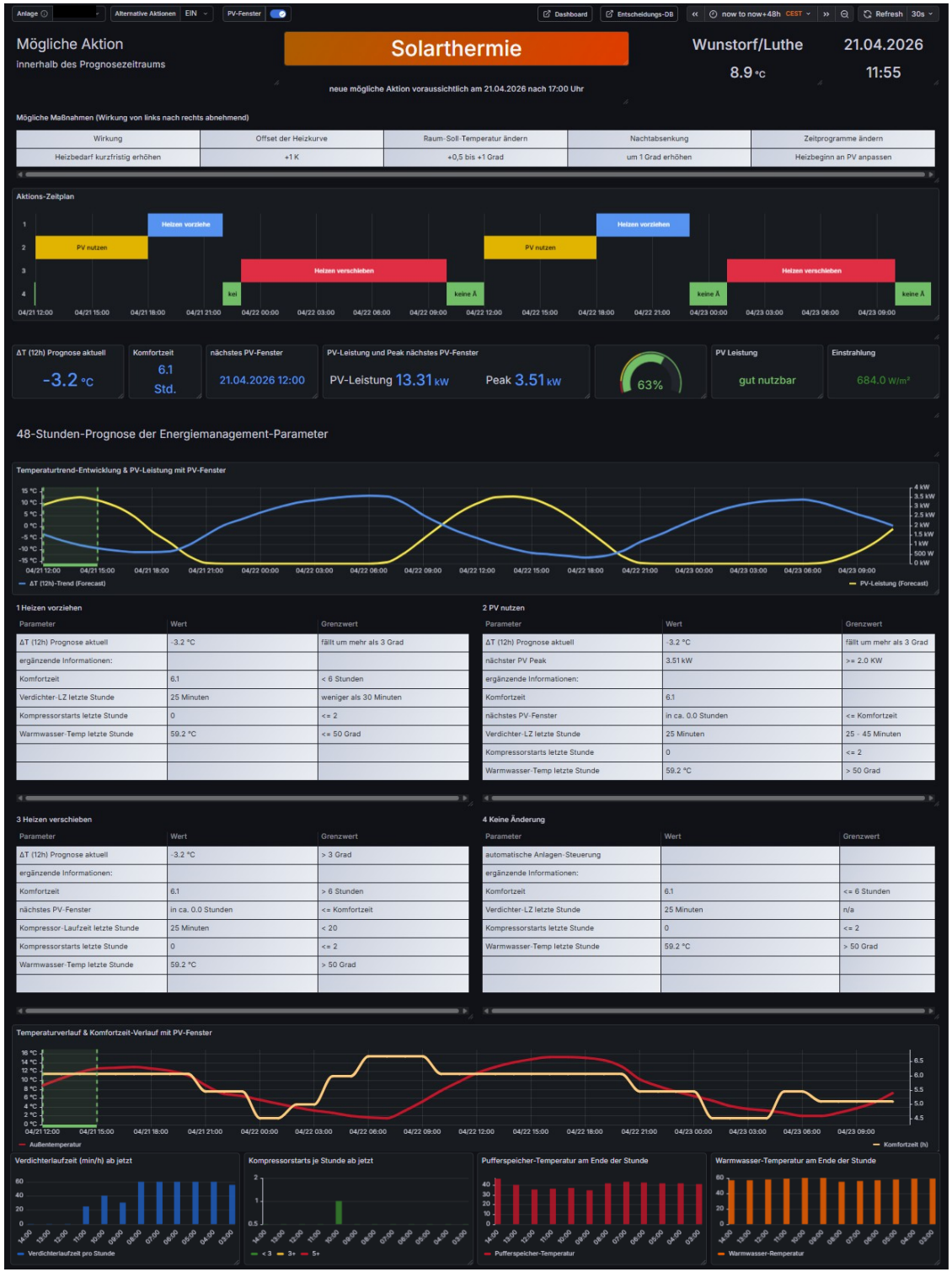
Entscheidungs-Dashboard für Viessmann-Wärmepumpen



eizungs- & PV-Entscheidungsdashboard
Search...
ctrl+k
Edit
Export
Share
rnative Aktionen
EIN
PV-Fenster
Dashboard
Entscheidungs-DB
now to now+48h
CEST
Refresh
Solarthermie
24.04.2026
14.0 °C
14:49
eitraums
neue mögliche Aktion voraussichtlich am 24.04.2026 nach 17:00 Uhr
von links nach rechts abnehmend)
Heizen vorzieh
PV nutzen
Heizen verschieben
keine Änder
keine Änder
Heizen vorzieh
PV nutzen
Heizen verschieben
keine Ä
keine Ä
04/24 21:00
04/25 00:00
04/25 03:00
04/25 06:00
04/25 09:00
04/25 12:00
04/25 15:00
04/25 18:00
04/25 21:00
04/26 00:00
04/26 03:00
04/26 06:00
04/26 09:00
04/26 12:00
Komfortzeit
5.1 Std.
nächstes PV-Fenster
24.04.2026 15:00
PV-Leistung und Peak nächstes PV-Fenster
PV-Leistung 9.21 kW
Peak 2.94 kW
60%
PV Leistung
gut nutzbar
Einstrahlung
654.0 W/m²
se der Energiemanagement-Parameter
PV-Leistung mit PV-Fenster
3.5 kW
3 kW
2.5 kW
2 kW
1.5 kW
1 kW
500 W
0 kW
04/24 21:00
04/25 00:00
04/25 03:00
04/25 06:00
04/25 09:00
04/25 12:00
04/25 15:00
04/25 18:00
04/25 21:00
04/26 00:00
04/26 03:00
04/26 06:00
04/26 09:00
04/26 12:00
PV-Leistung (Forecast)

Überblick

- Vorausschauende Wärme-pumpensteuerung
- Stündlich aktualisierte Live-Wetterdaten
- 48h Prognosezeitraum
- Heizungsfokus (Heizkreis und Warmwasser)
- Temperaturtrend (Δ12h)- und PV-gesteuert
- Nächstes nutzbares PV-Fenster
- Komfortzeit zur Entscheidungs-absicherung
- Zusätzliche relevante Betriebsdaten zur Entscheidungsunterstützung
- 4 Aktions-Szenarien
- Solarthermie-Berücksichtigung
- Handlungsvorschläge zur Umsetzung



Vorbemerkung

„Für das Vergangene gibt der Kaufmann nichts.“ - Eugen Schmalenbach, Die Werte von Anlagen und Unternehmungen in der Schätztechnik, Zeitschrift für handelswirtschaftliche Fragen (ZfhF) 1917/1918, S. 1 ff.

Dieser Satz bringt Dilemma und Grenzen der betriebswirtschaftlichen Forschung gleichermaßen auf den Punkt. Denn nur die Zukunft lässt sich gestalten. Alle vorliegenden Dashboards zur Wärmepumpenanalyse und -steuerung (inkl. dem vorstehend dargestellten) kranken gleichermaßen daran, dass sie bestenfalls die aktuellen Zustände berücksichtigen. Um beim Betrieb einer Wärmepumpe jedoch Geld zu sparen (insbes. Stromverbrauch, Verschleiß), bedarf es einer vorausschauenden Berücksichtigung von Live-Wetterdaten, da Umgebungstemperatur und Sonnenstrahlung technisch bedingt die maßgeblichen Parameter für die technische und ökonomische Effizienz aller Wärmepumpen sind. Anbieter wie NIBE oder Stiebel-Eltron haben dies bereits erkannt und lassen diesbezügliche Prognosewerte angabegemäß in ihre Wärmepumpensteuerung einfließen. Die Viessmann-VICare-App berücksichtigt derzeit noch keine Wetterprognose-Daten.

Rein sachlogisch ist die Berücksichtigung von Wetterprognosedaten ein Software-Thema für die Steuer-Apps (VICare). Solange dies nicht der Fall ist, aber auch später zu Funktions-Kontrollzwecken, kann ein unabhängiges Dashboard helfen, die eigene Wärmepumpe optimal zu betreiben. Dies ist die Motivation für die Entwicklung des nachstehend beschriebenen Dashboards. Es stellt bewusst eine **robuste, minimalistische Optimierungsregel** dar, mit der **ca. 90% des möglichen Einsparpotenzials** erzielt werden können - und die mittels der VICare-App problemlos in der Praxis umgesetzt werden kann:

- Stündlich abgerufene Wetterdaten von Meteo-Open (die Aktualität hängt natürlich vom Anbieter ab)
- Einen Prognosezeitraum von 48h
- Die fortgeschriebene Temperaturentwicklung in den nächsten 12h je Stunde im Prognosezeitraum sowie die Prognose der Strahlungsintensität (Radiation) in diesem Zeitraum
- Die individuelle Leistungsfähigkeit der vorhandenen PV-Anlage (KWp, PV-Leistungsfaktor) sowie die Nutzbarkeitsschwelle der PV-Leistung im Hinblick auf den Betrieb der Wärmepumpe (4-Stunden-Prognosefenster)
- Ergänzende Verschleißparameter (Kompressor-Laufzeit und Anzahl Verdichterstarts) und Betriebsdaten (Warmwasser-Temperatur) in den letzten 12 Stunden
- Konzentration auf 4 grundlegende Aktions-Szenarien zur Gewährleistung der Handhabbarkeit der vorgeschlagen Aktionen mittels der VICare-App.

Entscheidend für den praktischen Nutzen dieses Dashboards ist in jedem Fall die Umsetzung der vorgeschlagenen Aktionen im Alltag,

Die verarbeiteten Daten stehen in einer separaten Datenbank (entscheidungs_dashboard.db), die ergänzend für die Beurteilung der vorgeschlagenen Aktion verwendeten Verschleiß- und Betriebsdaten werden synchron (stündlich) aus der viessmann_events.db (vieventlog) gelesen.

Entscheidungs-Dashboard - Überblick

Ein-/Ausblenden aller 4 Aktions-Szenarien
(alle oder nur aktiver Vorschlag)

aktueller Aktionsvorschlag
oder Solarthermie

Mögliche Maßnahmen
(Umsetzung) → aktiver Vorschlag

Aktions-Zeitplan
(GANTT-Diagramm)

aktuelle Prognosewerte

nächstes (nutzbares)
PV-Fenster

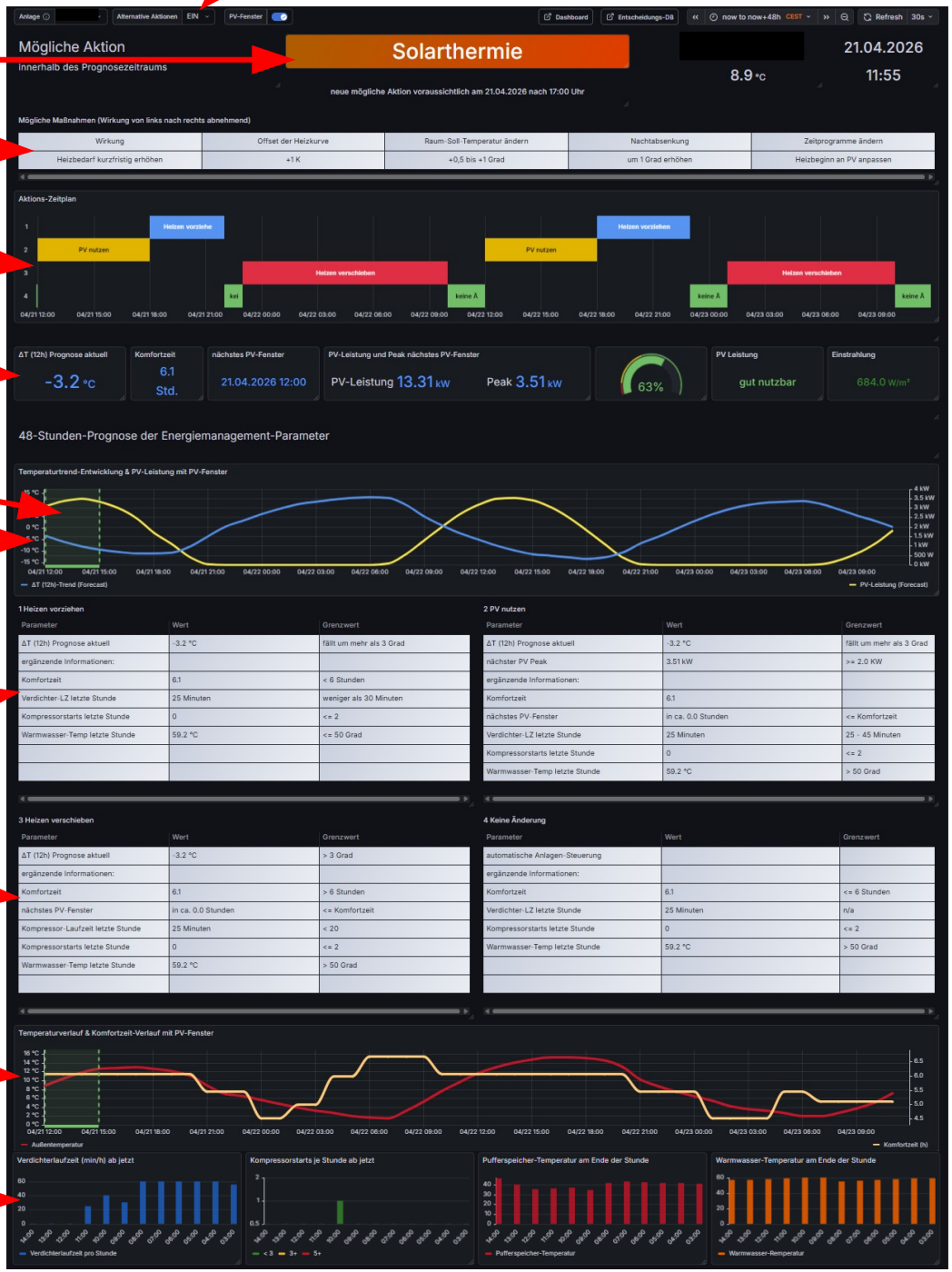
Zeitverlauf Wettertrend
& und PV-Leistung
(48h-Prognose)

4 grundlegende Aktions-Szenarien
→ Entscheidungsfindung

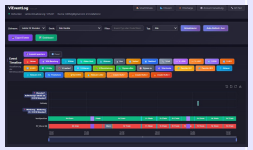
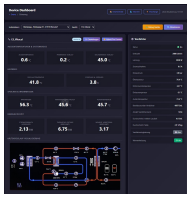
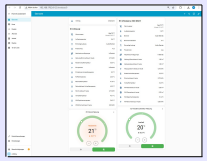
- 1 Heizen vorziehen
- 2 PV nutzen
- 3 Heizen verschieben
- 4 Keine Änderung

Ist-Temperaturverlauf
& Komfortzeitverlauf
mit nächstem nutzbaren
PV-Fenster
(48h-Prognose)

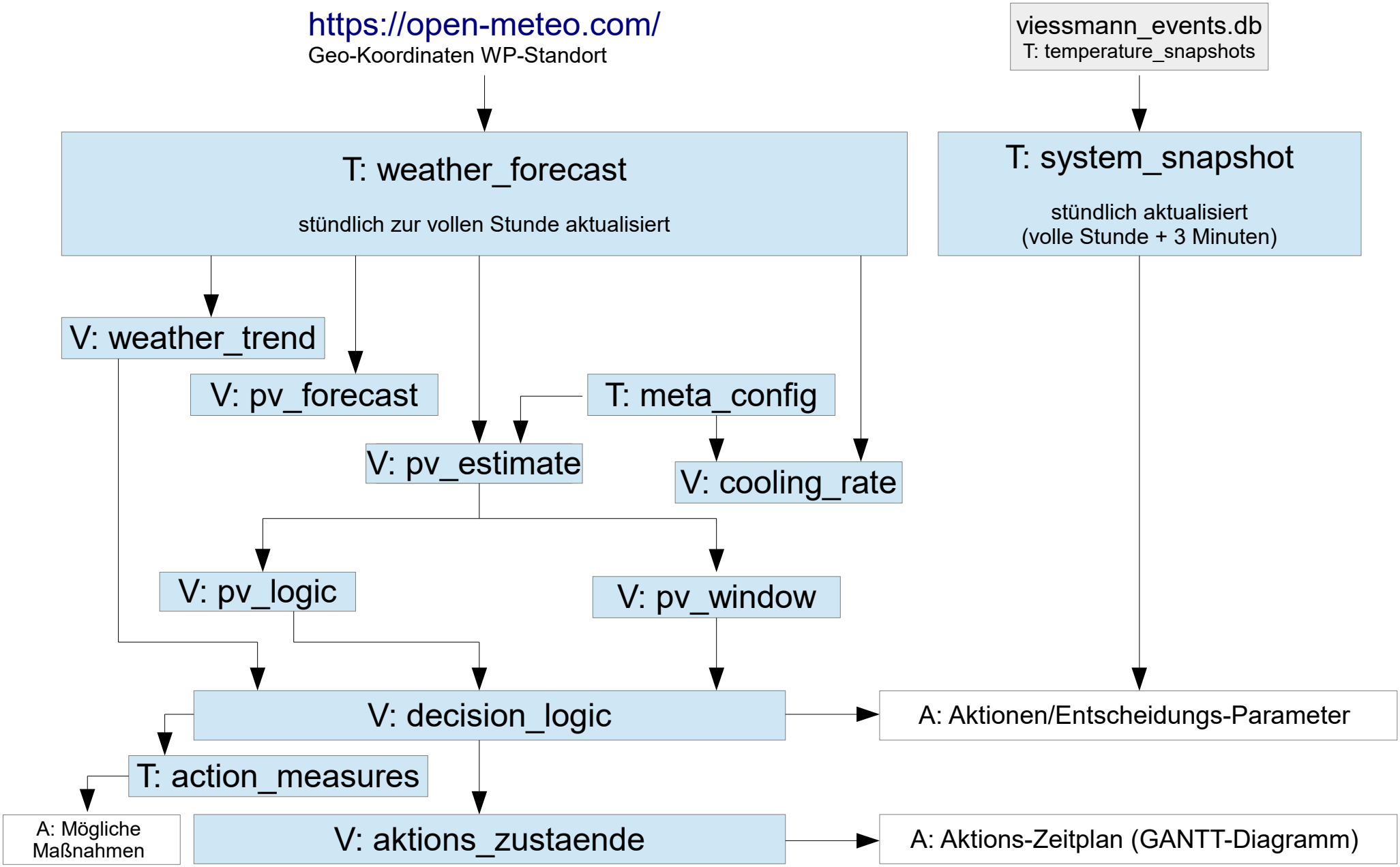
relevante Verschleiß-/
Betriebs-/Komfortdaten
→ Entscheidungsabsicherung



Use Case

Einstiegsbildschirme	bedarfsweise Vertiefung	Funktion	Aufruf (Windows/Synology)
Entscheidungs-Dashboard		Entscheidungs-Dashboard: als Einstieg in die Wärmepumpensteuerung Zukunftsorientierte Steuerung der Wärmepumpe (Wetterdaten, PV) **Achtung**: vieventlog muss (permanent) laufen für Datenbank-Update	Entscheidungs-Dashboard Aufruf Dashboard: • Localhost:3000 (Windows) • synlology_ip:**3030** (Synology) Achtung: Port gem. Portmapping im Grafana-Container
Alternatives Dashboard		Dashboard: als Einstieg in vieventlog Zentrales Monitoring für Wärmepumpe **Achtung**: vieventlog **muss** (permanent) laufen für Datenbank-Update	Dashboard Aufruf Dashboard: • Localhost:3000 (Windows) • synlology_ip:**3030** (Synology) Achtung: Port gem. Portmapping im Grafana-Container
	vieventlog/events 	Vieventlog/Events Accountverwaltung Steuerung API-Calls für Datenbank Datenbank-Update Detail-Informationen zu Events Verbindung zu Smart Client, Vitovent, Vitocharge	Vieventlog/events Aufruf per Link aus Dashboard über • Localhost:5000 (Windows-PC) • synlology_ip:**5050** (Synology) Achtung: Port gem. Portmapping im vieventlog-Container (Synology)
	vieventlog/Dashboard (Kältekreislauf) 	Vieventlog/Dashboard: API-Calls Einstellungen vieventlo/dashboard Geräte Debug (API-Variable) Kältekreislauf Heizkurve Sensorwerte (API) Steuerung VICare-Werte Statistik	Vieventlog/events Aufruf per Link aus Dashboard über • Localhost:3000 (Windows-PC) • synlology_ip:**5050**/dashboard (Synology) Achtung: Port gem. Portmapping im vieventlog-Container (Synology)
	VICare Homeassistent 	VICare Homeassistant VICare-Daten für PC/Synology Anzeige Daten/Sensorwerte wie über VICare-AP VICare-Alternative auf Windows-PC	VICare HomeAsssistant → muss zusätzlich installiert werden Aufruf per Link aus Dashboard über • Windows-PC: nicht installiert, über Synology • synlology_ip:**8123** Achtung: Port gem. Portmapping im vieventlog-Container (Synology)

Struktur der entscheidungs_dashboard.db



T = Tabelle V = View A = Anzeige im Dashboard

Erzeugung der entscheidungs_dashboard.db (1)

T: meta_config

```
CREATE TABLE meta_config  (
  key TEXT PRIMARY KEY,
  value REAL
```

Initialwerte:.0045);

```
INSERT INTO meta_config VALUES ('pv_threshold', 2.0);
INSERT INTO meta_config VALUES ('pv_kwp', 4.9)eta_config VALUES ('pv_threshold', 2.0);
INSERT INTO meta_config VALUES ('pv_kwp', 4.9)
INSERT INTO meta_config VALUES ('temp_verlust_gebaeude', 0.29)
INSERT INTO meta_config VALUES ('comfort_time_reference_value', 6.0)
INSERT INTO meta_config VALUES ('solar_threshold', 450.0)
INSERT INTO meta_config VALUES ('solar_buffer_threshold', 48.0)
INSERT INTO meta_config VALUES ('min_solar_buffer_threshold', 36.0)
INSERT INTO meta_config (key, value, text_value) VALUES ('installation_id', 1, '12134567')
```

Update (Beispiele):

```
UPDATE meta_config set value = 0.0065 WHERE key = 'pv_factor'
UPDATE meta_config set value = 2.1 WHERE key = 'pv_threshold'
UPDATE meta_config set value = 4.8 WHERE key = 'pv_kwp'
UPDATE meta_config set value = 0.28 WHERE key = 'temp_verlust_gebaeude'
UPDATE meta_config set value = 5.0 WHERE key = 'comfort_time_reference_value'
UPDATE meta_config set value = 430.0 WHERE key = 'solar_threshold'
UPDATE meta_config set value = 47.0 WHERE key = 'solar_buffer_threshold'
UPDATE meta_config set value = 42.0 WHERE key = 'min_solar_buffer_threshold'
UPDATE meta_config set text_value = 'NNNNNNN' WHERE key = 'installation_id'
```

T: weather_forecast

```
CREATE TABLE IF NOT EXISTS weather_forecast (
   timestamp TEXT PRIMARY KEY,
   temp_now REAL,
   temp_3h REAL,
   temp_6h REAL,
   temp_12h REAL,
   radiation REAL,
   wind_now,
   wind_6h,
   wind_12h
);
```

T: system_snapshot

```
CREATE TABLE "system_snapshot" (
        "timestamp_mez"  TEXT NOT NULL,
        "installation_id"  TEXT NOT NULL,
        "compressor_starts_1h" INTEGER NOT NULL,
        "runtime_minutes_1h" REAL NOT NULL,
        "dhw_temp" REAL,
        "buffer_temp"  REAL,
        PRIMARY KEY("timestamp_mez","installation_id")
)
```

V: pv_forecast

```
CREATE VIEW pv_forecast AS
SELECT
   w.timestamp,
   w.radiation,
   ROUND(
      w.radiation / 1000.0 * m.value,
   2) AS pv_kw
FROM weather_forecast w
JOIN meta_config m
  ON m.key = 'pv_kwp';
```

V: pv_estimate

```
CREATE VIEW pv_estimate AS
SELECT
   timestamp,
   radiation,
   ROUND(MIN (
   radiation *
     (SELECT value FROM meta_config WHERE key='pv_factor'),
  (SELECT value FROM meta_config WHERE key='pv_kwp') , 2)
)
FROM weather_forecast
```

Erzeugung der entscheidungs_dashboard.db (2)

V: pv_window

```
CREATE VIEW pv_window AS
SELECT
   timestamp,
   pv_kw,

   MAX(pv_kw) OVER (
      ORDER BY timestamp
      ROWS BETWEEN CURRENT ROW AND 3 FOLLOWING
   ) AS pv_next_peak

FROM pv_estimate;
```

V: decision_logic

```
CREATE VIEW decision_logic AS

SELECT
   w.timestamp,
   w.delta_12h,
   p.pv_kw,
   p.radiation,
   pw.pv_next_peak,

   CASE
      WHEN w.delta_12h < -3 AND pw.pv_next_peak >=
         (SELECT value FROM meta_config WHERE key='pv_threshold')
         THEN 'HEAT_MIDDAY'

      WHEN w.delta_12h < -3
         THEN 'HEAT_NOW'

      WHEN w.delta_12h > 3
         THEN 'REDUCE'

      ELSE 'STABLE'
   END AS action

FROM weather_trend w
JOIN pv_estimate p ON w.timestamp = p.timestamp
JOIN pv_window pw ON w.timestamp = pw.timestamp;
```

V: pv_logic

```
CREATE VIEW pv_logic AS
SELECT
   timestamp,
   pv_kw,

   CASE
      WHEN pv_kw >= (SELECT value FROM meta_config WHERE key='pv_threshold') THEN 1
      ELSE 0
   END AS pv_usable,

   CASE
      WHEN pv_kw >= (SELECT value FROM meta_config WHERE key='pv_threshold') THEN 'GOOD'
      WHEN pv_kw >= 0.5 THEN 'WEAK'
      ELSE 'NONE'
   END AS pv_quality

FROM pv_estimate;
```

Erzeugung der entscheidungs_dashboard.db (3)

V: aktions_zustaende

```
CREATE VIEW aktions_zustaende AS
WITH ordered AS  (
  SELECT
      timestamp,
  action,
  LAG(action) OVER (ORDER BY datetime(timestamp)) AS prev_action
    FROM decision_logic
),
  phase_mark AS (
    SELECT
      timestamp,
      action,
      CASE
         WHEN action <> prev_action OR prev_action IS NULL THEN 1
         ELSE 0
      END AS new_phase
    FROM ordered
),
  phase_id AS (
    SELECT
      timestamp,
      action,
      SUM(new_phase) OVER (ORDER BY datetime(timestamp)) AS phase
    FROM phase_mark
),
  phase_start AS (
    SELECT
      phase,
      MIN(timestamp) AS time_start,
  action
    FROM phase_id
    GROUP BY phase,
  action
),
```

Fortsetzung

```
  intervals AS (
    SELECT
      datetime(time_start) AS time_start,
      COALESCE(
         LEAD(datetime(time_start)) OVER (ORDER BY datetime(time_start)),
         datetime((SELECT MAX(timestamp) FROM decision_logic), '+1 hour')
      ) AS time_end,
      action
    FROM phase_start
)

SELECT

    CASE
      WHEN datetime(time_start) < datetime('now','localtime')
      THEN datetime('now','localtime')
      ELSE datetime(time_start)
    END AS time_start,
  time_end,
  CASE
      WHEN action = 'HEAT_NOW' THEN '1 HEAT_NOW'
      WHEN action = 'HEAT_MIDDAY' THEN '2 HEAT_MIDDAY'
      WHEN action = 'REDUCE' THEN '3 REDUCE'
      WHEN action = 'STABLE' THEN '4 STABLE'
    END AS metric,
  action,
  (select text from meta_config where key = 'installation_id') AS installation_id

FROM intervals

WHERE datetime(time_end) >= datetime('now','localtime')

ORDER BY datetime(time_start);
```

Erzeugung der entscheidungs_dashboard.db (4)

T: action_measures

```
CREATE TABLE action_measures (
    action TEXT KEY NOT NULL,
    Text1,
    content1,
    Text2,
    content2,
    Text3,
    content3,
    Text4,
    content4,
    Text5,
    content5
)
```

V: weather_trend

```
CREATE VIEW weather_trend AS
SELECT
    timestamp,
    temp_3h  - temp_now AS delta_3h,
    temp_6h  - temp_now AS delta_6h,
    temp_12h - temp_now AS delta_12h,

    CASE
        WHEN (temp_12h - temp_now) < -3 THEN 'COLD'
        WHEN (temp_12h - temp_now) >= 3 THEN 'WARM'
        ELSE 'STABLE'
    END AS trend_class

FROM weather_forecast;
```

V: cooling_rate

```
CREATE VIEW cooling_rate as
SELECT timestamp,

 (select value from meta_config where key = 'temp_verlust_gebaeude')
*
CASE
    WHEN max(wind_now, wind_6h) <   5 THEN 1.0
    WHEN max(wind_now, wind_6h) < 15 THEN 1.1
    WHEN max(wind_now, wind_6h) < 25 THEN 1.3
    WHEN max(wind_now, wind_6h) < 40 THEN 1.5
    ELSE 1.7
  END
  *
  CASE
    WHEN (21 - max(temp_now, temp_6h)) < 5 THEN 0.7
    WHEN (21 - max(temp_now, temp_6h)) < 10 THEN 0.9
    WHEN (21 - max(temp_now, temp_6h)) < 15 THEN 1.0
    WHEN (21 - max(temp_now, temp_6h)) < 20 THEN 1.2
    ELSE 1.4
  END
as cooling_rate

FROM weather_forecast;
```

weather_import.py (1)

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import requests
import sqlite3
from datetime import datetime, timedelta
import certifi
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

# ----------------------------
# CONFIG
# ----------------------------
LAT = 52.4238
LON = 9.4359
BASE_DIR = "/volume1/docker/entscheidungs_dashboard"
DB_FILE = os.path.join(BASE_DIR, "db", "entscheidungs_dashboard.db")
HOURS_FORECAST = 48

# ----------------------------
# Setup
# ----------------------------
os.makedirs(os.path.dirname(DB_FILE), exist_ok=True)

# ----------------------------
# Open-Meteo API
# ----------------------------
ENDPOINT = (
    f"https://api.open-meteo.com/v1/forecast?"
    f"latitude={LAT}&longitude={LON}"
    f"&hourly=temperature_2m,shortwave_radiation,wind_speed_10m"
    f"&timezone=Europe/Berlin"
)

session = requests.Session()
retry = Retry(
    total=5,
    connect=5,
    read=5,
    backoff_factor=1,
    status_forcelist=[500, 502, 503, 504],
)
adapter = HTTPAdapter(max_retries=retry)
session.mount("https://", adapter)

resp = session.get(ENDPOINT, timeout=30, verify=certifi.where())

if resp.status_code != 200:
    print(f"ERROR: API request failed with status {resp.status_code}")
    exit(1)

data = resp.json()

times = data["hourly"]["time"]
temps = data["hourly"]["temperature_2m"]
rads  = data["hourly"]["shortwave_radiation"]
winds = data["hourly"]["wind_speed_10m"]

# ----------------------------
# DB
# ----------------------------
conn = sqlite3.connect(DB_FILE)
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS weather_forecast (
    timestamp TEXT PRIMARY KEY,
    temp_now REAL,
    temp_3h REAL,
    temp_6h REAL,
    temp_12h REAL,
    radiation REAL,
    wind_now REAL,
    wind_6h REAL,
    wind_12h REAL
)
""")

# ----------------------------
# Cleanup
# ----------------------------
now = datetime.now()
cur.execute("DELETE FROM weather_forecast WHERE timestamp < ?", (now.isoformat(),))
conn.commit()

# ----------------------------
# Insert forecast
# ----------------------------
for i, ts_str in enumerate(times):
    ts = datetime.fromisoformat(ts_str)

    if ts > now + timedelta(hours=HOURS_FORECAST):
        break

    temp_now = temps[i]

    # Temperatur-Trends
    temp_3h  = temps[i+3]  if i+3 < len(temps)  else temp_now
    temp_6h  = temps[i+6]  if i+6 < len(temps)  else temp_now
    temp_12h = temps[i+12] if i+12 < len(temps) else temp_now

    rad_v = rads[i] if i < len(rads) else 0.0

    # ----------------------------
    # WIND (3 Ebenen)
    # ----------------------------
    wind_now = winds[i] if i < len(winds) else 0.0

    wind_6h = (
        sum(winds[i:i+6]) / 6
        if i+5 < len(winds)
        else wind_now
    )

    wind_12h = (
        sum(winds[i:i+12]) / 12
        if i+11 < len(winds)
        else wind_6h
    )
```

weather_import.py (2)

```
  # ------------------------------
    # INSERT
    # ------------------------------
    cur.execute("""
        INSERT OR REPLACE INTO weather_forecast
        (timestamp, temp_now, temp_3h, temp_6h, temp_12h,
         radiation, wind_now, wind_6h, wind_12h)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        ts_str,
        temp_now,
        temp_3h,
        temp_6h,
        temp_12h,
        rad_v,
        wind_now,
        wind_6h,
        wind_12h
    ))

conn.commit()
conn.close()

print(f"OK: imported forecast for next {HOURS_FORECAST} hours")
```

system_snashot.sh

```
#!/bin/bash

BASE_DIR="/volume1/docker/entscheidungs_dashboard"
TARGET_DB="$BASE_DIR/db/entscheidungs_dashboard.db"
SOURCE_DB="/volume1/docker/vieventlog/db/viessmann_events.db"
LOG="$BASE_DIR/skripte/logs/system_snapshot.log"

echo "---- $(date) START system_snapshot ----" >> "$LOG"

sqlite3 "$TARGET_DB" >> "$LOG" 2>&1 <<EOF

ATTACH DATABASE '$SOURCE_DB' AS src;

WITH

-- aktuelle Stunde als Stundenende (für den Datensatz)
hw AS (
    SELECT datetime(strftime('%Y-%m-%d %H:00:00', 'now','localtime')) AS ts
),

-- Kompressorstarts aggregiert pro Stunde
t AS (
    SELECT
        installation_id,
        datetime(strftime('%Y-%m-%d %H:00:00', timestamp_mez), '+1 hour') AS ts,
        SUM(CASE WHEN compressor_ein_aus = 1 THEN 1 ELSE 0 END) AS starts
    FROM src.temperature_snapshots_takte_heute
    WHERE installation_id=1234567'
      AND timestamp_mez >= datetime('now','localtime','-2 hour')
      AND timestamp_mez < datetime('now','localtime')
    GROUP BY installation_id, ts
),

-- Laufzeit aggregiert pro Stunde
s AS (
    SELECT
        installation_id,
        datetime(strftime('%Y-%m-%d %H:00:00', timestamp_mez), '+1 hour') AS ts,
        SUM(CASE WHEN compressor_active = 1 THEN COALESCE(sample_interval,0) ELSE 0 END) AS runtime_min
    FROM src.temperature_snapshots_stat
    WHERE installation_id='1234567'
      AND timestamp_mez >= datetime('now','localtime','-2 hour')
      AND timestamp_mez < datetime('now','localtime')
    GROUP BY installation_id, ts
),

--- letzter Temperaturwert der Stunde (DHW + Buffer)
temp_last AS (
    SELECT
        installation_id,
        datetime(strftime('%Y-%m-%d %H:00:00', timestamp_mez), '+1 hour') AS ts,
        dhw_temp,
        buffer_temp
    FROM (
        SELECT
            installation_id,
            timestamp_mez,
            dhw_temp,
            buffer_temp,
            ROW_NUMBER() OVER (
                PARTITION BY datetime(strftime('%Y-%m-%d %H:00:00', timestamp_mez), '+1 hour')
                ORDER BY timestamp_mez DESC
            ) AS rn
        FROM src.temperature_snapshots_stat
        WHERE installation_id='1234567'
          AND timestamp_mez >= datetime('now','localtime','-2 hour')
          AND timestamp_mez < datetime('now','localtime')
    )
    WHERE rn = 1
)

-- Datensatz schreiben
INSERT OR REPLACE INTO system_snapshot (
    timestamp_mez,
    installation_id,
    compressor_starts_1h,
    runtime_minutes_1h,
    dhw_temp,
    buffer_temp
)
SELECT
    hw.ts AS timestamp_mez,
    '1234567' AS installation_id,
    COALESCE(t.starts,0) AS compressor_starts_1h,
    COALESCE(s.runtime_min,0) AS runtime_minutes_1h,
    temp_last.dhw_temp,
    temp_last.buffer_temp
FROM hw
LEFT JOIN t ON t.ts = hw.ts AND t.installation_id='1234567'
LEFT JOIN s ON s.ts = hw.ts AND s.installation_id='1234567'
LEFT JOIN temp_last ON temp_last.ts = hw.ts AND temp_last.installation_id='1234567';

DETACH DATABASE src;

EOF

if [ $? -eq 0 ]; then
  echo "$(date) OK: snapshot updated" >> "$LOG"
else
  echo "$(date) ERROR: snapshot failed" >> "$LOG"
fi

echo "---- $(date) END system_snapshot ----" >> "$LOG"
```

Aufgaben im im Synology-Aufgabenplaner / CRON-Jobs

Wetterdaten

Allgemein:

Aufgabe: Wetterdaten
Benutzer: Hans-Hermann

Zeitplan:

Wiederholen: Täglich
Start: 00:00
Wiederholen: jede Stunde
Letzte Ausführungszeit: 23:00

Aufgabestellung:

Benutzerdefiniertes Skript

```
echo "---- $(date) weather_import start ----"
>> /volume1/docker/entscheidungs_dashboard/skripte/logs/weather_import.log

/usr/local/bin/python3.9 /volume1/docker/entscheidungs_dashboard/skripte/
weather_import.py
>> /volume1/docker/entscheidungs_dashboard/skripte/logs/weather_import.log 2>&1
```

Systemdaten

Allgemein:

Aufgabe: Systemdaten
Benutzer: root

Zeitplan:

Wiederholen: Täglich
Start: 00:03
Wiederholen: jede Stunde
Letzte Ausführungszeit: 23:03

Aufgabestellung:

Benutzerdefiniertes Skript

```
/volume1/docker/entscheidungs_dashboard/
skripte/system_snapshot.sh
```

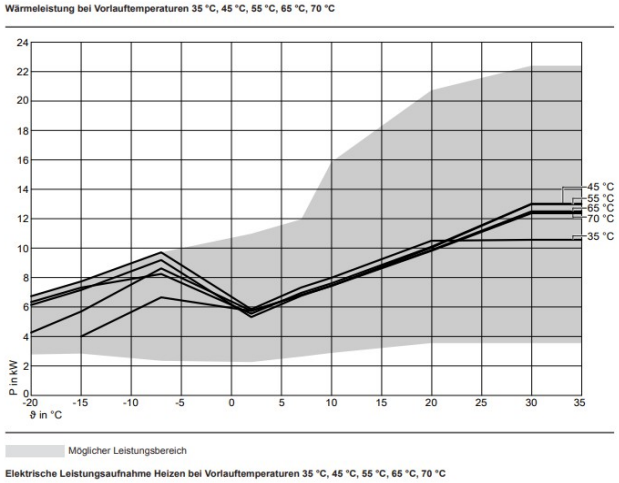
Konzeptioneller Ansatz (1)

Eine Wärmepumpe ist eine im Vergleich zu fossilen Brennwertgeräten vergleichsweise träge Heizquelle. Ihre (technische und ökonomische) Effizienz im Einsatz wird maßgeblich durch Umgebungstemperatur und angeforderte Wärmeleistung beeinflusst. Der Verschleiß (und damit die Lebensdauer) einer Wärmepumpe werden wiederum maßgeblich durch Laufzeit sowie Anzahl und zeitliche Abfolge der Kompressorstarts beeinflusst. Die Effizienz wird zusätzlich beeinflusst durch die Nutzung von Photovoltaik (Strompreis) sowie von Solarthermie (Stromverbrauch, Laufzeit, Kompressorstarts).

Das maximal mögliche Einsparpotenzial wird zu rd. 90% durch die Umgebungstemperatur und die Effizienzfaktoren determiniert.

Die erforderliche Wärmeleistung bei bestimmten Umgebungstemperaturen wird über die Einstellung der Heizkurve, die gewünschten Solltemperaturen für Heizung und Warmwasser über direkte Einstellungen am Gerät oder die VICare-App gesteuert.

Die Trägheit der Wärmepumpe bedingt, dass die Eingriffe pro Zeiteinheit (Stunde) rein sachlogisch begrenzt sind. Auch müssen die notwendigen Anpassungsmaßnahmen ausschließlich über die Eigensteuerung (VICare-App, Anlagendisplay) möglich sein. Und die Ableitung der Aktionsvorschläge muss auch für den technisch weniger interessierten Nutzer einfach verständlich und nachvollziehbar sein.



Hinzu kommt in meinem Fall, dass die modernen baulichen Rahmenbedingungen die technisch bedingte Trägheit der Wärmepumpe eher unterstützen:

- Gebäudemasse
- niedrige Heizlast
- Solarthermie
- Inverter-WP (Vitocal 250 A)
- große (Flach-) Heizkörper / Fußbodenheizung

Aus diesen Gründen habe ich mich bei diesem Dashboard für eine **robuste, minimalistische Optimierungsregel** entschieden, mit der **ca. 80-90% des möglichen Einsparpotenzials** erzielt werden können. Professionelle Temperaturmanagement Systeme basierend auf komplexen Modellen sind zwar wesentlich genauer, kommen aber am Ende überwiegend zu den gleichen Empfehlungen. Aufgrund der Trägheit bringen komplexe Echtzeitoptimierungen oft kaum Vorteile.

Konzeptioneller Ansatz (2)

Konzeptioneller Ansatz

Konzeptionell verfolgt das Dashboard einen physikalisch-hierarchischen Ansatz:

1	Gebäudephysik	→ Temperaturtrend (ΔT 12h)	→ Aktionsvorschlag zur Entscheidung
2	Energiequelle	→ nächstes PV-Fenster	
3	Komfortzeit	→ max. akzeptierte Gebäude-Auskühlung	→ ergänzende Informationen zur Absicherung der Entscheidung
4	Anlagenzustand	→ Verdichterlaufzeit, Kompressorstarts	
5	Komfort	→ Warmwasser-Temperatur	

Die **Optimierung** basiert auf einem 2-stufigen Ansatz:

❑

Grundlage: *stabile* Heizkurve, grundsätzlich ganzjährig tagsüber passend

↓ **daraus abgeleitet**

1. Basisoptimierung der Anlage (optional): Generische, langfristige **saisonale Betriebsprofile**[1)]
→ Stabilität: Verringerung von Änderungsfrequenz und Reduzierung der Parameter-Veränderungen

↓ **situativ modifiziert durch**

2. kurzfristige Optimierungen: Temporäre, **kurzfristige Aktionsprofile** als Vorschläge aus dem Dashboard
→ additive Abweichungen von den saisonalen Betriebsprofilen

1) Die Einstellungen der Anlagen-Grundstrategie sind unabhängig vom Dashboard festzulegen.

Konzeptioneller Ansatz (3)

<u>Grundsatz</u>:

1. (einfacher, robuster) Algorithmus *schlägt vor*
2. Nutzer *entscheidet*, was wirklich machbar und sinnvoll ist bzw. wieviel Spielraum tatsächlich besteht

<u>Entscheidungs-Parameter</u>

- Temperaturtrend (ΔT 12h) → primäres Entscheidungskriterium

  Trend zur vorausschauenden Erkennung von Temperaturänderungen und Berücksichtigung der Gebäudeträgheit gegenüber Temperaturveränderungen

  12h-Zeitraum zur Glättung von Fehlern in der Wettervorhersage

- PV-Leistung → kein primäres Entscheidungskriterium, sondern ergänzendes Kriterium bei tatsächlichem Heizungsbedarf

  nächstes PV-Fenster, wenn in einem 4-Stunden-Zeitraum ab jetzt das nächste Mal die pv_threshold einmal überschritten wird

<u>Ergänzende Informationen</u>

- Komfortzeit: um wieviel Grad darf das Gebäude max. auskühlen und nach welcher Zeit wird die Grenze erreicht

  → primär wichtig bei Verschieben des Heizzyklus („Heizen verschieben“ und „auf PV warten“)
  → Indikator für die Dringlichkeit beim Vorziehen des Heizzyklus

- Warmwassertemperatur → beeinflusst Vorziehen und Verschiebendes Heizzyklus (Komfort)

- Kompressorstarts (Verschleiß-Parameter) → Anzahl sollte durch Veränderung des Heizzyklus nicht über Obergrenze erhöht werden

- Kompressor-Laufzeit (Verschleiß-Parameter)

  → sollte durch Veränderung des Heizzyklus nur erhöht werden, wenn eine bestimmte Laufzeit noch nicht erreicht ist
  → mögliche Begrenzung bei Vorziehen des Heizzyklus

Konzeptioneller Ansatz (4)

Entscheidungs-Logik

- **Entscheidungspriorisierung:** → Komfort schlägt Effizienz

  → Merksatz: Erst verhindern wir Komfortprobleme, dann reagieren wir auf's Wetter, dabei nutzen wir PV

  1. Komfort (außerhalb Algorithmus, da individuell-situative Einschätzung)
  2. Temperaturtrend
  3. PV-Optimierung

- **Stabilitätszone** → Temperaturtrend (ΔT 12h) zwischen -3 Grad und +3 Grad → keine Veränderung

  → verhindert (zusätzliches) Takten
  → verhindert dauernde Strategiewechsel
  → verhindert Forecast-Rauschen

Gesamtübersicht

Temperaturtrend	Wirkung	Komfortzeit	PV	Aktion
> + 3 °C	Kein vorzeitiger Heizbedarf	ausreichend	innerhalb Komfortzeit	Heizen verschieben (bis PV eintritt)[1)]
	Kein vorzeitiger Heizbedarf	ausreichend	außerhalb Komfortzeit	Heizen verschieben (max. bis Komfortzeitende erreicht)
	Kein vorzeitiger Heizbedarf	zu kurz	egal	keine Änderung
-3 °C bis + 3 °C	egal	egal	egal	keine Änderung
< -3 °C	vorzeitiger Heizbedarf	ausreichend	Innerhalb Komfortzeit	Auf PV warten[2)]
	vorzeitiger Heizbedarf	ausreichend	außerhalb Komfortzeit	Keine Änderung
	vorzeitiger Heizbedarf	zu kurz	egal	Heizen vorziehen

1) gleiche Wirkung, aber Altion „Heizen verschieben“, weil kein vorzeitiger Heizbedarf;
2) Aktion „auf PV warten“, weil vorzeitiger Heizbadarf

Konzeptioneller Ansatz (5)

<u>Berechnung der Komfortzeit</u> → KI-generiert

- **Durchschnittliche Abkühlrate des Gebäudes ohne Heizung pro Stunde**

  ⟶ Die Abkühlrate hängt hauptsächlich vom Transmissionswärmeverlust der Gebäudehülle (U-Werte von Wand, Dach, Fenster) und vom Luftwechsel ab. Je schlechter die Dämmung, desto schneller geht Wärme verloren.

Baujahr / energetischer Zustand	Typische Abkühlrate ohne Heizung
Altbau vor 1950 (ungedämmt)	1,0 bis 2,0 °C/h
1950 - 1977 (vor Wärmeschutzverordnung)	0,6 bis 1,2 °C/h
1978 bis 1995 (erste Dämmstandards)	0,3 – 0,7 °C/h
1996–2009 (WSVO / frühe EnEV)	0,2 – 0,4 °C/h
1976, 2008 erweitert (→ teilsaniert)	0,2 – 0,4 °C/h (0,29 °C/h)
2010–heute (EnEV / GEG Neubau)	0,05 – 0,2 °C/h
Passivhaus	0,02 – 0,05 °C/h

- **Temperaturfaktor** → Temperaturdifferenz (21 °C minus Temperatur in 2m Höhe)

Temperaturdifferenz (21 °C minus Temperatur in 2m Höhe)	Faktor
< 5 °C	0,7
>= 5 °C und < 10 °C	0,9
>= 10 °C und < 15 °C	1,0
>= 15 °C und < 20 °C	1,2
>= 20 °C	1,4

- **Windfaktor** → Windgeschwindigkeit 10 m über dem Boden
  → Wärmeübergang außen (Konvektion) und Luftinfiltrationen (Undichtigkeiten)

Windgeschwindigkeit (10 m über dem Boden	Faktor
< 5 m	1,0
>= 5 m und < 15 m	1,1
>= 15 m und < 25 m	1,3
>= 15 m und < 40 m	1,5
>= 40 m	1,7

prinzipielle cooling_rate =

Typische Abkühlrate / h * Temperaturfaktor * Windfaktor

↓

cooling_rate =

Typische Abkühlrate / h *
(21 – max(temp_now, temp_6h) *
max(wind_now, wind_6h)

⟶ ungünstigste Temperatur- und Windannahmen
→ höchstmögliche cooling_rate
→ **niedrigstmögliche Komfortzeit**

→ Eliminierung von Prognose-Ungenauigkeiten
→ Risikominimierung für Komfortverlust

↓

Komfortzeit =

$$\frac{\text{Typische Abkühlrate / h * comfort_time_reference_value}}{\text{cooling_rate}}$$

Konzeptioneller Ansatz (6)

Solarthermie

→ verläuft weitestgehend komplementär zur PV-Leistung
→ ist ohne Sensoren individuell nur sehr schwer zu prognostizieren (u.a. solare Einstrahlung, Kollektortemperatur, Gesamtwirkungsgrad, Dachausrichtung, Dachneigung, Bewölkung/Verschattung, Rohr-/Speicherverluste, Startverluste)
→ Solarthermie-Ertrag abhängig von Radiation, Gesamtwirkungsgrad **und Systemzustand** (tatsächliche Pufferspeicher-Temperatur)
→ Puffertemperatur: „Ist der Puffer noch kühl genug, damit Solar sinnvoll einspeisen kann?“

==> Prognose der Solarthermie nur eingeschränkt möglich, da neben der Einstrahlung auch der Systemzustand (Puffertemperatur) entscheidend ist
==> Solarthermie bildet keinen eigenständigen Aktionsvorschlag sondern kann nur **aktuellen Aktions-Vorschlag temporär overrulen**

==> pauschale Berücksichtigung (Pufferspeicher-Temperaturgrenzen. Strahlungs-Intensität (W/m$^{2)}$)
→ „Radiation entscheidet, ob Solar möglich ist – der Pufferspeicher entscheidet, ob es im System sinnvoll ist.“

- **Solare Einstrahlung** → KI-generiert

Radiation	Qualität (Nutzbarkeit) → Anzeige „Einstrahlung“
< 250 W/m^2	schlecht
250 – 400 W/m2	möglich
400 – 650 W/m2	gut
> 650	sehr gut

technische Startschwelle für wirksame Solarthermienutzung
→ „solar_threshold“ in Tabelle meta_config

- **Pufferspeicher-Temperatur (Systemzustand)**

→ „Ist der Pufferspeicher noch kalt genug, um Solarenergie aufzunehmen“
→ hoch genug, dass Solar noch sinnvoll laden kann
→ niedrig genug, um ineffiziente Nachladung zu vermeiden

→ Es gibt unterschiedliche (Ober-) Grenzen bei Pufferspeicher-Temperatur für Heizung und Warmwasser (wenn Frischwassermodul über Pufferspeicher)

Puffterspeicher-Temperatur	Solarverhalten	Bedeutung
>= 35 °C	Optimaler Solarbereich	Wärmepumpe aus, Pufferspeicher übernimmt
40 – 48 °C	guter Arbeitsbereich	Solarthermie für Heizungsbetrieb
> 48 °C	abnehmender Wirkungsgrad	zunehmend ineffizient für Heizungsbetrieb, aber effiziente Nutzung für Warmwasser
> 60 °C	oft Stagnation (technische Temperaturgrenze)	Nur für Warmwasser thermische „Reserve“ für kalte Tage
> 75 °C	Mögliche Stagnation bei voller Beladung	

Untergrenze für wirksame Heizungswirkung
→ „min_solar_buffer_threshold“ in Tabelle meta_config

Obergenze für wirksame Heizungswirkung
→ wenn *nur* für Heizungsbetrieb
→ „solar_buffer_threshold“ in Tabelle meta_config

↓

auf (technischen) **Maximalwert** setzen, wenn Pufferspeicher auch für Warmwasser genutzt wird (z.B. 85 °C)

Erläuterung der Vorgehensweise (1)

Das Dashboard

- basiert auf Wetter-Livedaten – Prognosehorizont jeweils 48 Stunden, stündlich aktualisiert;
- Berücksichtigt dabei die Temperaturdifferenz jetzt und in12 Stunden (∆12h Trend);
- Leitet die an die tatsächliche PV-Installation angepasste prognostizierte PV-Leistung aus der Sonnenstrahlung sowie der maximalen Leistung der PV-Anlage (in kwp) und einem „pv_factor“ ab, der die *standardisierte* Umsetzung der Sonnenstrahlung auf die installierte PV-Kollektoren in elektrische Energie wiedergibt:

  PV-Leistung = Minimum
  - *aus* radiation / 1000 * pv_factor *und*
  - maximale PV-Leistung (kwp);

- zeigt zusätzlich Kompressorlaufzeit, Anzahl Kompressorstarts und der Warmwassertemperatur der letzten 12 Stunden jeweils zur vollen Stunde an;
- ist auf der Grafana-Plattform entwickelt, d.h. es erlaubt jedem Nutzer die Anpassung an seine indivusuellen Gegebenheiten.

Die Steuerdaten (pv_factor, pv_threshold, pv_kwp, temp_verlust_gebäude und comfort_time_reference_value) können per einfachem INSERT/UPDATE in der Tabelle „meta_config“ eingestellt werden.

Bei der zeitlichen Abgrenzung gibt es bei einer Datengranularität auf Stundenbasis zwei Möglichkeiten: bis zur aktuellen Stunde (Tageszeit) oder ab der Stunde, mit der die Daten in der Datenbank enthalten sind (Timestamp). Ich habe mich für einen Mischansatz entschieden:

- Die Daten für Kompressorlaufzeit, Kompressorstarts und Warmwassertemperatur können naturgemäß jeweils bis zur vollen Stunde angezeigt werden (Ist-Daten).
- Bei den Prognosedaten (∆12h Wettertrend) sowie der PV-Leistungsprognose werden mit Ablauf einer Stunde (Tageszeit) die Werte der nächsten Stunde (Datenbank angezeigt (+1h-Strategie). Dies erscheint nach Vergleich mit den Tages-Temperaturen bei uns realitätsnah. Bei der Identifizierung des nächsten PV-Fensters wird zudem der Trägheit der Wärmepumpe Rechnung getragen.

Aus den Daten werden folgende Werte jeweils aktuelle (= zur nächsten vollen Stunde, vgl. vorstehend) bereitsgestellt

- Trend ∆ T (12h) Prognose
- Durchschnittlicher Temperaturverlust des Hauses pro Stunde
- Max. Komfortzeit (→ ergibt sich aus max. akzeptierter Auskühlung des Hauses und durchschnittl. Temp.-Verlust d. Gebäudes)
- nächstes PV-Fenster (nutzbare 4h mit mind. einmal innerhalb der 4 Stunden erreichtem PV-Peak > pv-threshold)
- PV-Leistung und Peak im nächsten PV-Fenster
- PV-Leistung (% Kompressorleistung und Nutzbarkeit)

Erläuterung der Vorgehensweise (2)

Viele Optimierungsmodelle versuchen zusätzlich zu berücksichtigen:

- Gebäudemasse (Gebäudephysik)
- Pufferspeicherzustand (Temperatur)
- Heizkurve
- Vorlauf/Rücklauf
- Strahlung
- Interne Gewinne (alle wärmenden Quellen, die nicht von der Heizung stammen, z.B. Menschen im Raum, elektrische Geräte, Haushaltsaktivitäten Solarertrag über Fernseher)

Hiervon ist m.E. die **Gebäudephysik** am Wichtigsten. Sie bestimmt letztlich, wie weit (zeitlich) ein Heizvorgang vor- oder nachverlagert werden kann, damit die Wärmeverluste/-gewinne nicht höher sind als der Einspareffekt (Stromverbrauch, PV-Ertrag). Ein freistehendes Haus aus den 70er-Jahren, teil-modernisiert verliert ohne Heizung typischerweise 0,2 bis 0,3 Grad pro Stunde. Damit liegt die maximale Heizverschiebung Komfortzeit) bei etwa 6 Stunden (ca. 1,75 Grad).

Die **Pufferspeichertemperatur** sollte sinnvollerweise berücksichtigt werden, wenn sie steuerbar ist. Folgt sie praktisch nur dem Heiz-/Solarheizverlauf, ist die gespeicherte Energie im Heizbetrieb dagegen sehr gering. Wirkt der Pufferspeicher im Heizbetrieb eher als eine hydraulische Entkopplung von Sekundärkreis und Heizkreis, bringt er für die Heizentscheidung praktisch keinen Informationsgewinn.

Eine **stabile, d.h. ganzjährig tagsüber passende Heizkurve ist** unabdingbare *Voraussetzung* für stabile Optimierungsmaßnahmen.

Die **Sekundärvorlauftemperatur** zum Pufferspeicher sowie **Heizkreisvorlauf- und Rücklauf-Temperatur** sind Folgen der Variablen Heizkurve, Außentemperatur und Heizbedarf. Sie sind deshalb Resultate der Regelung, nicht aber deren Ursache, bringen mithin keine zusätzlichen Informationen, die nicht bereits in den Variablen enthalten sind.

Die **Strahlung** (Radiation) wird über die PV-Erträge berücksichtigt.

Die **internen Gewinne** sind in aller Regel inkonstant und wirken auch nicht über die gesamten Zeiträume der angemessenen Änderungsfrequenzen (Änderungsdauer mind. 2 Stunden).

Das Dashboard weist **4 Aktions-Vorschläge** aus:

- Heizen vorziehen
- PV nutzen
- Heizen verschieben
- Keine Änderung

Erläuterung der Vorgehensweise (3)

Der Ableitung der vorgeschlagenen Aktionen erfolgt ausschließlich Temperaturtrend- und PV-Prognose-abhängig (in der nachstehenden Tabelle **fett** dargestellt). Damit können rd. 90% des maximal möglichen Effizienzpotenzials gehoben werden. Für die Entscheidungssicherheit werden noch Zusatzinformationen bereitgestellt, damit die Systemstabilität gewährleistet bleibt.

Solarthermie habe ich bewusst raus gelassen, da die zwar in „Konkurrenz“ zur Wärmepumpe wirkt, aber eben komplementär zur PV. Sie könnte allenfalls in der Übergangszeit eine (geringe) unabhängige Auswirkung auf die vorgeschlagene Aktion haben, wenn die Sonne zwar wärmt, aber insgesamt nicht genug Kraft für einen hinreichenden PV-Peak hat. Diesen Effekt konnte ich nach 2 Wintern meiner WP-Nutzung nur an ganz wenigen Tagen und da auch nur für wenige Stunden um Mittag herum beobachten.

Entscheidungs-Parameter und ergänzende Informationen:

Parameter	Erläuterung	Heizen vorziehen	PV nutzen	Heizen verschieben	Keine Änderung
ΔT (12h) Prognose aktuell	**entscheidet ob Heizzyklus verschiebbar; maßgebliche Größe für ca. 80-90% des Einspar-potenzials**	**Fällt um mehr als 3 °C**	**Fällt um mehr als 3 °C**	**> 3 °C**	**Standardfall, wenn PV-Peak irrelevant und Temperaturtrend stabil** **Automatische Anlagen-Steuerung gem. eingestellter Heizkurve und Werte**
Nächster PV Peak	**WP braucht im Normalbetrieb 1 – 2 KW Strom**		**> 2,0 (gem. pv-threshold)**		
PV-Prognose (aktuell = per nächster voller h)	Ab 1 KW merkbarer aber nicht ausreichender Effekt			< 1 KW	n/a
Nächstes PV-Fenster	Lohnt es sich auf PV-zu warten	> 12 Stunden	<= 6 Stunden	<= 12 Stunden	n/a
Kompressorlaufzeit der letzten Stunde	Anti-Oszillation (zusätzlichem) Takten vorbeugen	Weniger als 30 Minuten	25 bis 45 Minuten	< 20 Minuten	n/a
Anzahl Kompressorstarts der letzten Stunde	Anti-Oszillation (zusätzlichem) Takten vorbeugen	<= 2	<= 2	<= 2	<= 2 (generell)
Warmwassertemperatur am Ende der letzten Stunde	Vermeidung von Störungen der Warmwasserzyklen	<= 50 °C (gem. Untergrenze WW-Temperatur)	n/a	> 50 °C	n/a

Umsetzung in der Praxis (1)

Das hinter dem Dashboard liegende Konzept basiert für die Umsetzung in der Praxis auf 2 Stufen (vgl. oben):

1. **Grundstrategie der Anlage**: Generische, langfristige saisonale Betriebsprofile (unabhängig vom und außerhalb des Dashboards)

2. **Situative Optimierungen (optional)**: Temporäre, kurzfristige Aktionsprofile als Vorschläge aus dem Dashboard → additive Abweichungen von den saisonalen Betriebsprofilen

Ausgangspunkt muss eine **stabile** Heizkurve sein,die das ganze Jahr tagsüber passt.

Die situativen Optimierungen sollten dabei aufgrund der Trägheit des Systems „Wärmepumpe“ und zur Vermeidung von Takten **begrenzt** werden:

- Offsetbereich der Heizkurve plus/minus 2 K vom Standardprofile
- Aktionen nur zulassen, wenn der Verdichter nicht läuft und die Anzahl der Verdichterstarts der letzten Stunde 1 überschreitet
- Aktionen mindestens 2 Stunden stabil lassen

1. Saisonale Standardprofile:

Saisonales Profil	Heizkurven Offset	Nachtabsenkung (Zeitraum)	Nachtabsenkung (Temperatur)	Warmwasser nachts
Winterbetrieb	0 K	22:00 bis 6:00 Uhr	17 Grad	Kann nachts gesperrt werden, bringt aber keine Vorteile
Übergangszeit **ohne** hinreichende PV-Unterstützung	-0,5 K	22:00 bis 6:00 Uhr	17 Grad	Keine WW-Aufbereitung
Übergangszeit **mit** hinreichender PV-Unterstützung	0 K	22:00 bis 6:00 Uhr	15 – 17 Grad	Kann nachts gesperrt werden, bringt aber keine Vorteile
Sommerbetrieb (Heizung im Abschaltbetrieb)	Heizung aus	n/a	n/a	Kann nachts gesperrt werden, bringt aber keine Vorteile

Übergangszeit bezeichnet je nach Witterung den Zeitraum Mitte September bis Ende November(in den letzten Jahren sogar bis Mitte Dezember) sowie den Zeitraum Mitte März bis Mitte Mai (dieses Jahr bereits ab Ende Februar mit hinreichender PV-Prognose).

Umsetzung in der Praxis (2)

2. Situative Optimierungen (kurzfristige Aktionen):

Die vier vom Dashboard alternativ vorgeschlagenen Aktionen verändern kurzfristig-situativ aufgrund der tatsächlichen Wettertrend- und PV-Prognosen die Einstellungen der Standardprofile. Diese Aktionen greifen lediglich temporär in den Basisbetrieb der Anlage ein, indem sie den Heizzyklus zeitlich verschieben.

			Alternativ (mit schwächerer Wirkung)		
Aktionsvorschlag	**Wirkung**	**Offset der Heizkurve**	**Raum-Soll-Temperatur ändern** (fast gleich gut wie Offset der Heizkurve)	**Nachtabsenkung** (schlechter, langsame Reaktionsgeschwinigkeit)	**Zeitprogramme ändern** (schlecht, grobe und sehr langsame Wirkung)
Heizen verschieben	Heizbedarf kurzfristig reduzieren	- 1 K	-0,5 bis -1 Grad	Absenkung um 1 Grad reduzieren	Heizbeginn später
PV nutzen	Heizbedarf kurzfristig erhöhen	+ 1 K	+0,5 bis +1 Grad	Absenkung um 1 Grad erhöhen	Heizbeginn früher (an PV anpassen)
Heizen vorziehen	Heizbedarf sofort erhöhen	+ 0,5 K	+ 1 Grad	Absenkung um 1 bis 2 Grad erhöhen	Heizbeginn früher
Keine Änderung	Heizbedarf unverändert lassen	0 K	0 Grad	Absenkung unverändert lassen	Keine Änderung

Dieser Ansatz nutzt die thermische Gebäudemasse, die Trägheit der Heizsysteme sowie zeitliche Verschiebbarkeit der Wärmeerzeugung. Deshalb reicht ein zudem sehr wirksamer Stellparameter (Offset der Heizkurve). Die gleichzeitige Steuerung über die diversen Komponenten der Smart-Home-Systeme (Raumthermostate, Pumpen, Ventile, Verdichter) erhöht den Regelungsaufwand und führt nach meiner Einschätzung dagegen möglicherweise zu instabilen Regelungen.

Zur Gewährleistung der Systemstabilität sollte

- der (Gesamt-) Offsetbereich auf plus/minus 2 K begrenzt werden;
- Aktionen nur zugelassen werden, wenn der Verdichter nicht läuft
- Aktionen mindestens 2 Stunden stabil gehalten werden.

Anpassen der Steuerungs-Daten (1)

Steuerdaten:

- PV-Einstellungen:
  → pv_factor, pv_threshold, pv_kwp → Tabelle „meta_config“

Komfortzeit

→ Stammdaten → temp_verlust_gebaeude, confort_time_reference_value → Tabelle „meta_config“
→ Temperaturfactor, Windfaktor → View „cooling_rate“

Solarthermie

→ Stammdaten → solar_threshold, solar_buffer_threshod, min_solar_buffer_threshold → Tabele meta_config“
→ Overruling aktuelle Aktion → Panel „Aktionsvorschlag“

Wetter-Trend

→ Bezeichnung in Anzeige → Panel „Trend“ → **Ignorierung** View „weather_trend“ !

temp_12 – temp_now <= -10 → deutlich kälter
temp_12 – temp_now <= -7 → kälter
temp_12 – temp_now <= -3 → deutlich kühler
temp_12 – temp_now <= -0,5 → kühler
temp_12 – temp_now <= 0,5 → stabil
temp_12 – temp_now <= 5 → wärmer
temp_12 – temp_now <= 10 → deutlich wärmer
temp_12 – temp_now <= 20 → erheblich wärmer
temp_12 – temp_now > 20 → heißer

PV-Fenster

- Zeitdauer für nächstes PV-Fenster
  → aktuelle + 3 folgende Stunden → View „pv_window“

- PV-Leistung
  → PV-Leistung (originär pv_kw)→ View „pv_forecast
  → PV-Leistung (angepasst an PV-Installation pv_kw) → pv_estimate
  → PV-Qualität → pv_usable, pv_quality ---> View “pv_logic“
  → PV-Leistungsgrad → Einstellungen im Leistungsgrad-Panel (ohne Titel) → Thresholds: grün = 2,0/pv_kwp, gelb = 0,5/pv_kwp

Anpassen der Steuerungs-Daten (2)

Aktionen

- Temperaturgrenzen (inkl. Stabilitätszone → implizit) → View „decison_logic“
- Aktions-Titel → HEAT_NOW, HEAT_MIDDAY, REDUCE, STABLE → View „decison_logic“
- Bezeichnung in Anzeige → Einstellungen im Entscheidungsvorschlags-Panel (ohne Titel) → Value mappings
  HEAT_NOW → Heizen vorziehen
  REDUCE → Heizen verschieben
  HEAT_MIDDAY → PV-Nutzen
  STABLE → Keine Änderungen

Entscheidung-Logik

- Einstellungen für Temperaturgrenzen für Entscheidungs-Auswahl → w.delta_12h → View „decision_logic
- PV-Peak (für Aktion „PV nutzen) → pv_threshold (Tabelle meta_config)

Maßnahmen je Aktionsvorschlag

→ Eintragungen in text/value-Paare → Tabelle „action_measures“
→ text = Kopfzeile, content = Wertzeile → beliebige Überschriften (text) und Inhalte (content)

Installation

Installation (Teil 1)

Die Installation des Entscheidungs-Dashboards läuft wie folgt ab:

1. Auslieferung:

- Dashboard-JSON zum Upload in Grafana
- entscheidungs_dashboard.db (SQLite-DB), bis auf Tabelle „meta_config“ ohne Inhalt
- Inhalt Tabelle „meta_config“: pv_factor, pv_threshold, pv_kwp, temp_verlust_gebaeude
- Jobs für Import der Wetterdaten und Systemparameter (aus viessmann_events.db von vieventlog)
- SQL-Statements für Tabellen/Views in der entscheidungs_dashboard.db zur Veränderung der Darstellung und Entscheidungslogik im Dashboard

2. Laden und konfigurieren des Entscheidungs-Dashboards in GRAFANA

→ Seiten 30 bis 37 dieser Datei

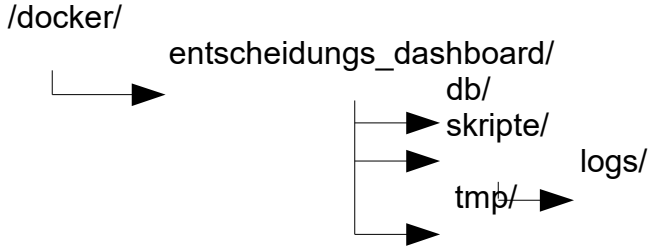
→ bei Anlage „New Datasource“ (vgl. Seite 19): Pfad zur entscheidungs_dashboard.db eintragen

→ Variablen bearbeiten (vgl. Seite 32):
- Variable „Anlage“: SQL-Statement in Query eintragen:
  select installation_id from temperature_snapshots group by installation_id
  alle anderen Felder unverändert lassen

→ Link „Dashboard“: http://-Link zum Alternativen Dashboard eintragen (falls installiert)

→ Link „Entscheidungs-DB“: hhtp://-Link zur entscheidungs_dashboard.db eintragen

3. Pfadstruktur für Datenbank, Skripte und Dateien anlegen (Beispiel: Synology)



Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (1) → Windows 11

1. Download und Installation

a) Download Grafana unter
https://grafana.com/grafana/download

b) Installation (nur Grafana OSS)

2. Grafana starten über Windows-Batch-Datei

→ „Grafana Start unter Windows.bat“
Achtung. Unbedingt als Administrator ausführen

→ ggf. Verzeichnis anpassen
Standard-Installation: Verzeichnis =

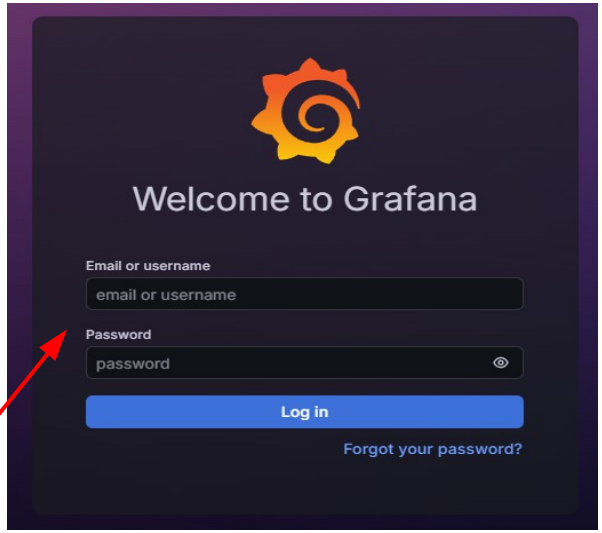
C:\Programme\GrafanaLabs\grafana\bin

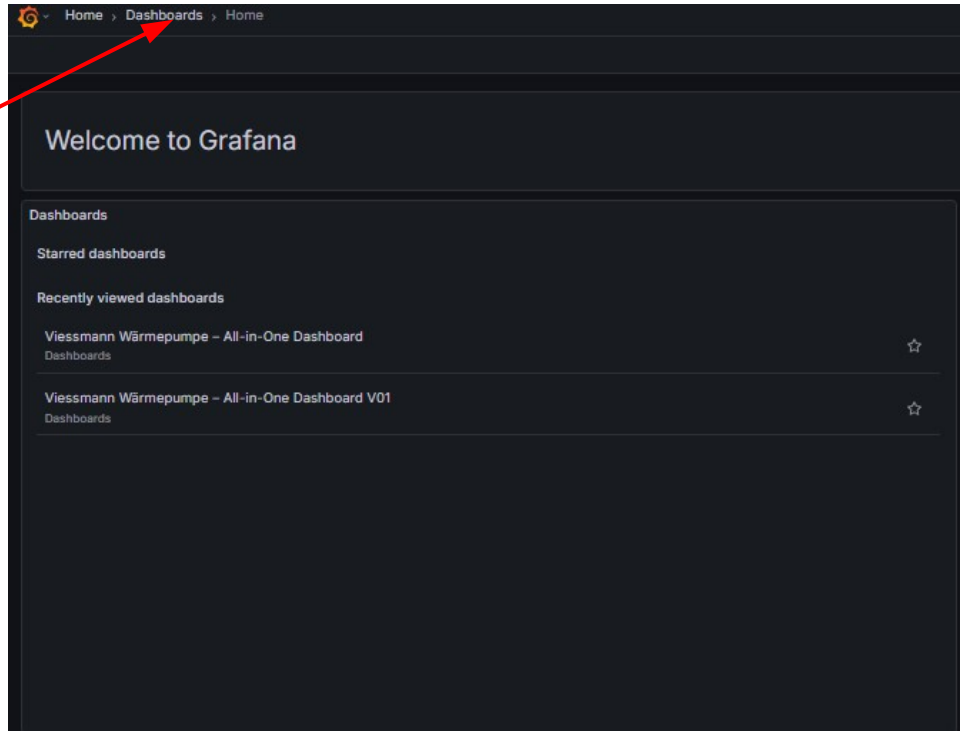
Datei = grafana-server.exe

Anmelden mit username = admin
Passwort = admin

Admin-Passwort ändern

auf Dashboard klicken



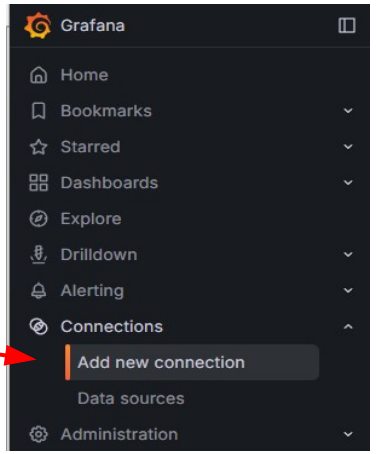


Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (2)

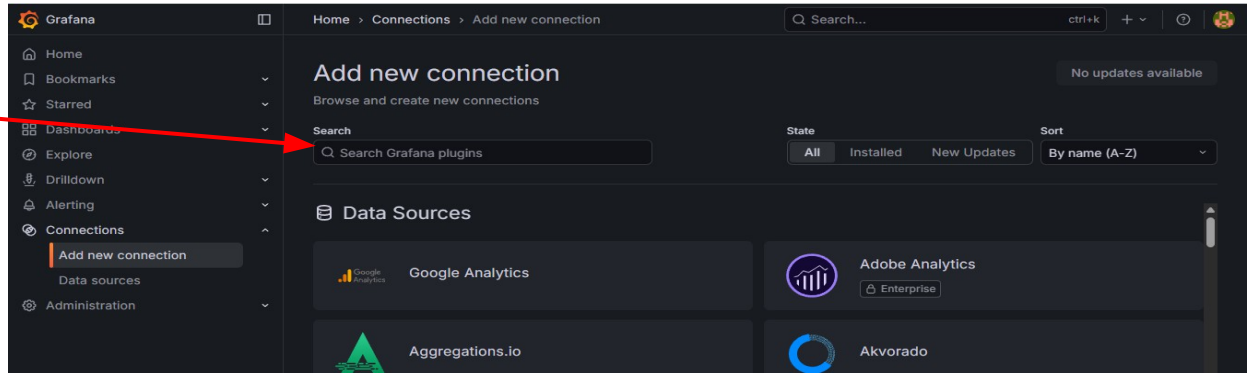
.

3. Datasource konfigurieren

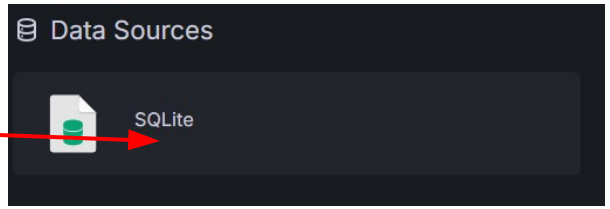
   → Add new connection



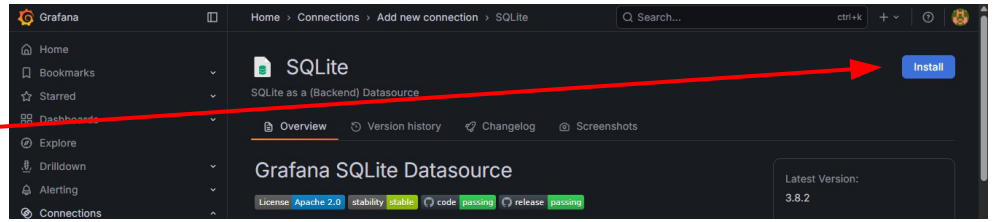
4. sqlite eingeben



5. auf SQLite klicken

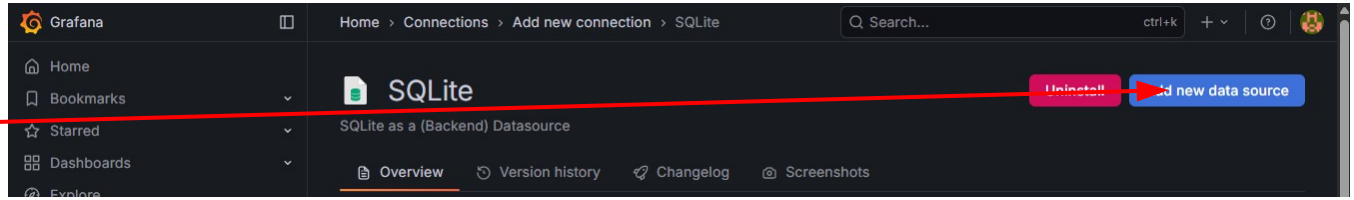


6. Auf Install klicken



Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (3)

7. auf Add new datasource klicken



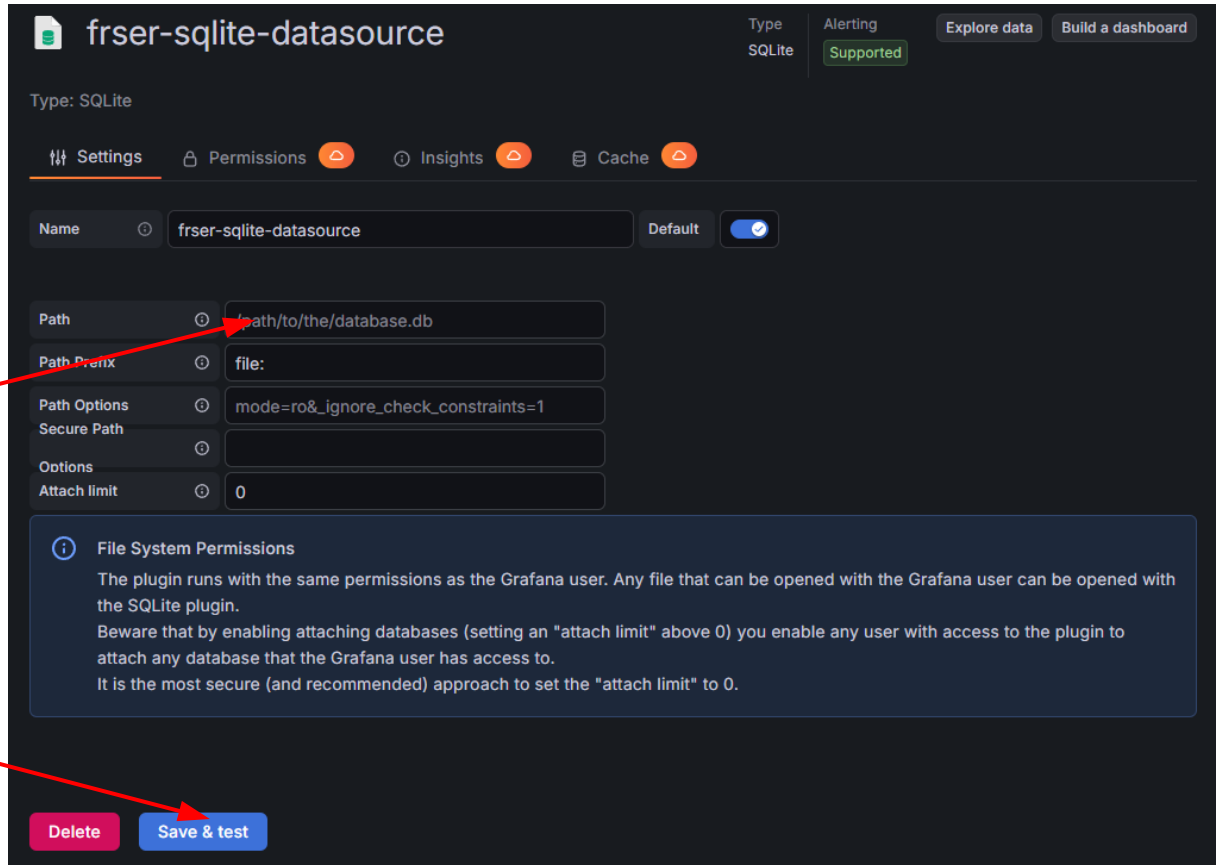
8. a) Pfad zur Datenbank eintragen

→ wo bei Euch die entscheidungs_dashboard.db physisch gespeichert ist

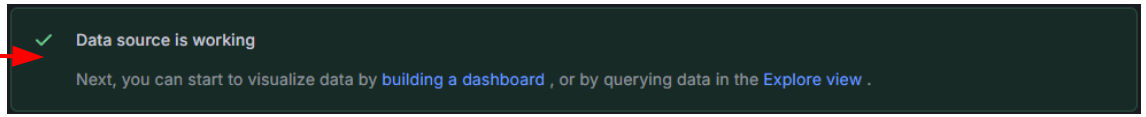
→ C:\ ….\entscheidungs-dashboad.de (Windows-Verzeichnis-Schreibweise)

→ alle übrigen Felder unverändert lassen

b) auf Save & Test klicken



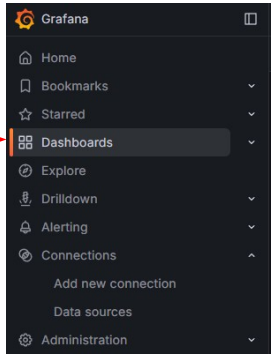
c) Erfolg!



Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (4)

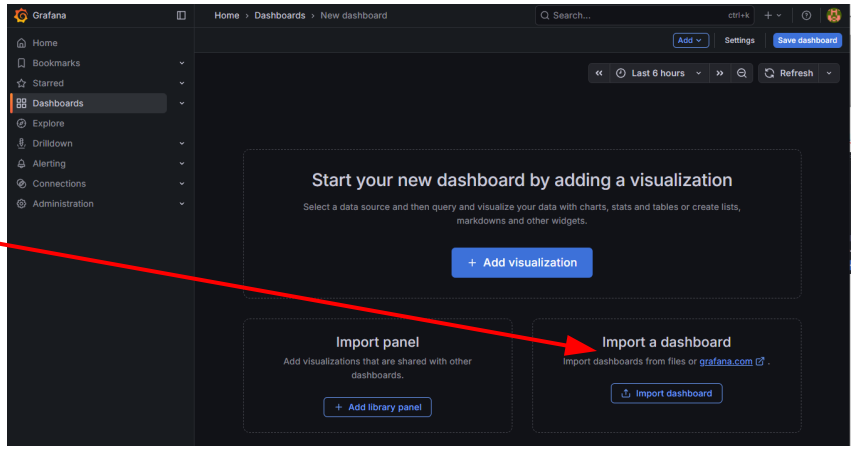
9. Dashboard anlegen

   → auf Dashboard klicken



auf + Create Dahboard klicken

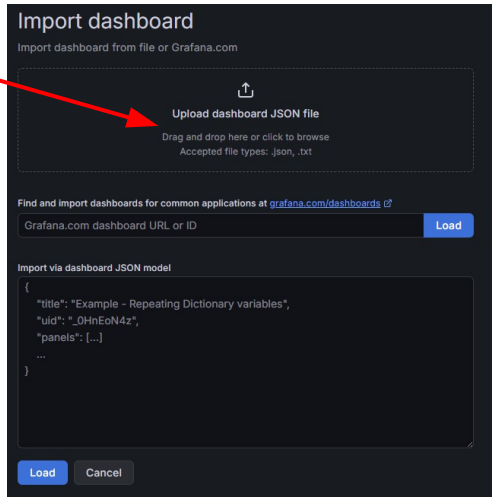
10. auf Import Dashboard klicken



11. auf Upload JSON-File klicken

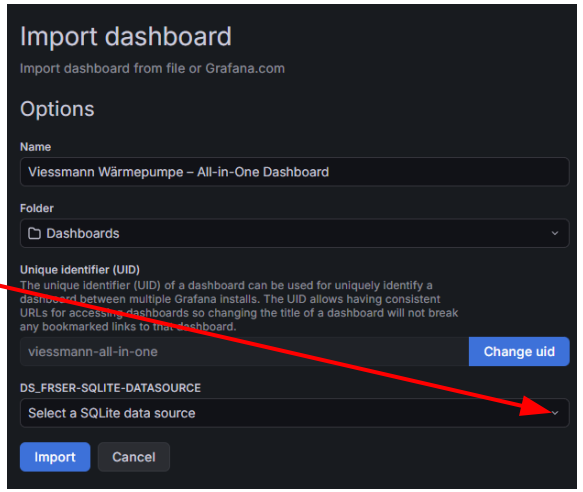
12. Herunterladen JSON-File aus GitHub

13. JSON-File über Windows-Explorer auswählen

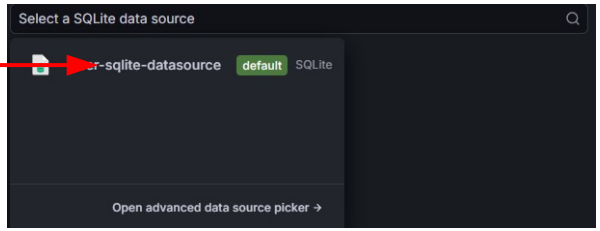


Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (5)

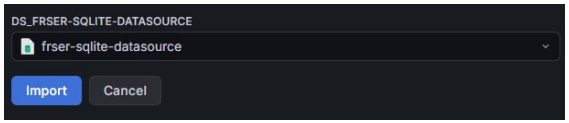
14. Datenbank auswählen



15. auf Frser-SQLite-datasource klicken

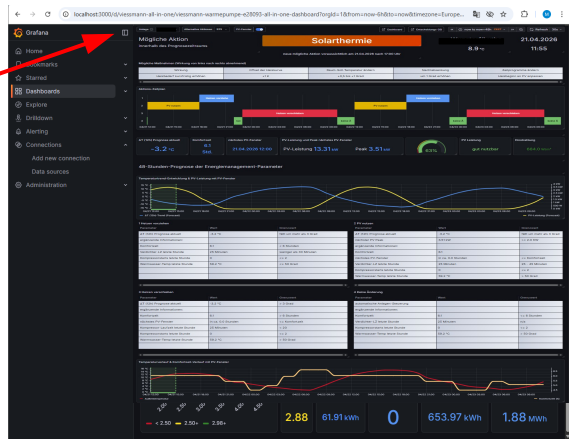


16. auf Import klicken



17. Dashboard wird geladen!

→ noch ohne Werte

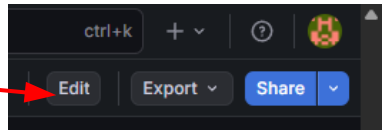


18. Grafana-Menü zuklappen

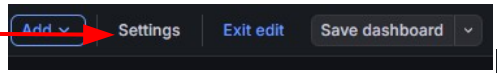
Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (6)

19. Anlagen einstellen

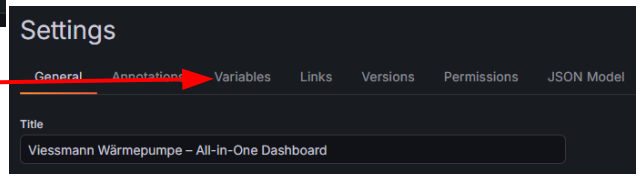
a) auf Edit (oben rechts klicken)



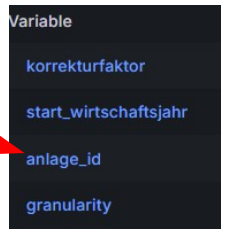
b) auf Settings klicken



c) auf Variables klicken



d) auf anlage_id klicken



e) in Query eingeben:

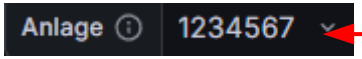
⟶ select installation_id from temperature_snapshots
group by installation_id

→ alle anderen Felder unverändert lassen

f) auf Save Dashboard klicken

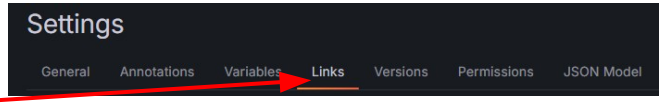
Laden und Konfigurieren des Entscheidungs-Dashboards in Grafana (7)

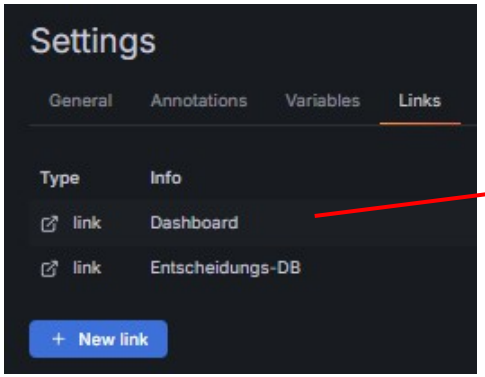
20. Anlage auswählen
    → zurück ins Dashboard
    → Anlage auswählen

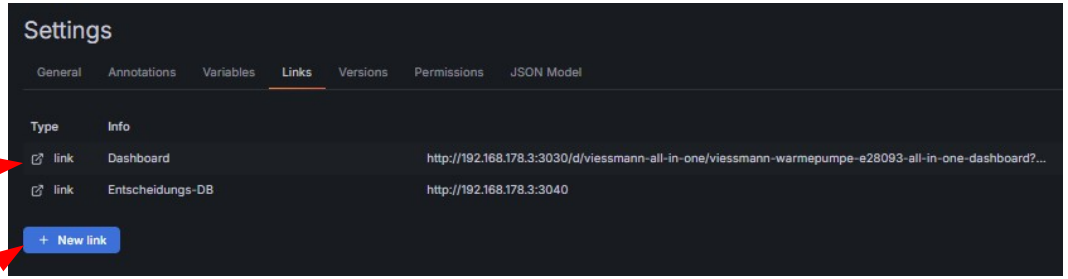


Hier stehen dann Eure Werte drin

21. Links einstellen oder löschen, wenn nicht gewünscht
    → auf Links klicken
    → URLs für gewünschte Links eingeben

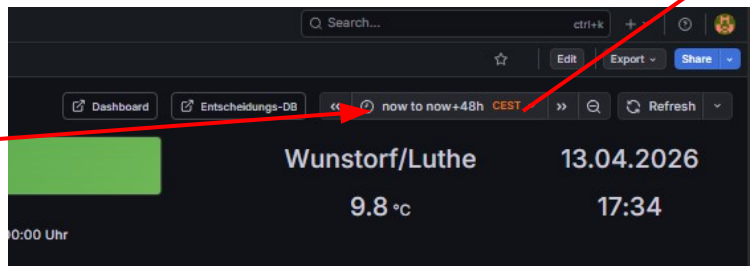


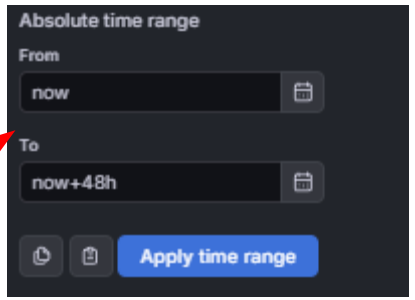




→ nicht gewünschte Links entfernen
    → ggf. neue Links aufnehmen
    → **auf Save Dashboard klicken !!!**
    → Links testen

22. Zeitzone und Intervall eingeben (MUSS!)
    → From: now
    → to: now+48h (ohne Leerzeichen)
    → alle Daten sollten vorhanden sein

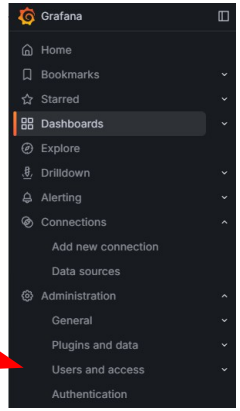




Nutzer und Berechtigungen anlegen

23. User und Berechtigungen anlegen

→ Menü öffnen



→ auf User und Access klicken

→ User und Berechtigungen festlegen

<u>Hinweis</u>:

Im Normalfall nur mit Usern ohne Admin-Rechte arbeiten, um ungewollte Veränderungen zu vermeiden

24. Anpassen/Ändern des Dashboards

→ bei Anmeldung mit Admin-Rechten kann das Dashboard beliebig verändert werden !

25. Windows-Batch-Datei zum Starten von Grafana im Hintergrund und Öffnen der Grafana-Oberfläche im Standard-Browser

→ Im zip-File ist neben dieser Einführung im pdf-Format noch eine Batch-Datei entalten, die

- Grafana im Hintergrund startet
- die Web.Oberfläche von Grafana im Standard-Browser öffnet.

<u>Achtung</u>: Die Batch-Datei MUSS als Administrator augeführt werden (rechte Maustaste → als Administrator ausführen)

Installation (Teil 2)

4. Datei „weather_import.py“ (Python) auf Festplatte erzeugen und konfigurieren – noch nicht starten

→ Konfiguration:

Geo-Positionsdaten der passenden Wetterstation eintragen:
→ auf https://open-meteo.com/en/docs?bounding_box=-90,-180,90,180 gehen
→ auf SEARCH klicken
→ Stadt (Wohnort) oder nächstgelegenen größeren Ort eingeben OK
→ LAT (Latitude) und LON (Longitude) in JOB eintragen

Pfad und Datenbanknamen eintragen

```
# ------------------
# CONFIG
# ------------------
LAT = 52.4238
LON = 9.4359
BASE_DIR = "/volume1/docker/entscheidungs_dashboard"
DB_FILE = os.path.join(BASE_DIR, "db", "entscheidungs_dashboard.db")
HOURS_FORECAST = 48
```

5. Python installieren (so noch nicht erfolgt) – mind. V3.9

→ Windows: https://www.python.org/downloads/windows/
→ weitere: hier
→ Synology: Paketzentrum → Community → SynoCommunity → Python 3.9 oder neuer auswählen und installieren

6. Job starten (Test)
→ in Protokolldatei prüfen, ob Download OK
→ in Datenbank Tabelle „weather_import“ prüfen, ob Einträge vorhanden sind

7. CRON-JOB bzw. Aufgabe in Aufgabenplaner (Synology) einrichten
→ tägliche Wiederholung, Update jede Stunde

8. entscheidungs_dashboard-Auslieferung.db laden
→ installation_id inTabelle meta_config eintragen
→ SQL-Statement: UPDATE meta_config set text_value = '**NNNNNNN**' WHERE key = 'installation_id'
mit NNNNNNN = eigene installations:id

Installation (Teil 3)

9. Datei „system_snapshot.sh“ auf Festplatte speichern und konfigurieren

```
#!/bin/bash

BASE_DIR="/volume1/docker/entscheidungs_dashboard"
TARGET_DB="$BASE_DIR/db/entscheidungs_dashboard.db"
SOURCE_DB="/volume1/docker/vieventlog/db/viessmann_events.db"
LOG="$BASE_DIR/skripte/logs/system_snapshot.log"
```

Pfade und Dateinamen eintragen
→ Base_DIR: übergordneter Pfad zu alllen Dateien des Entscheidungs-Dashboards
→ Target-DB = entscheidungs_dashboard.db
→ Source-DB = viesssmann_events.db
→ Log-Datei: system_snapshot.log

10. Job starten (Test)
→ Logfile prüfen, ob Download OK
→ DatenbankTabelle “system_snapshots“ prüfen, ob Einträge vorhanden sind

11. CRON-Job bzw. Aufgabe in Aufgabenplaner (Synology) einrichten

→ tägliche Wiederholung
→ stündlich zur Minute 03 wiederholen (z.B. 12:03) → damit sichergestellt ist,dass der letzte Satz der Stunde in viessmann:events.db enthalten ist

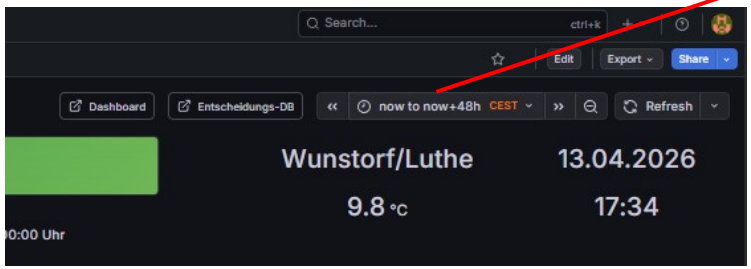
12. Dashboard starten

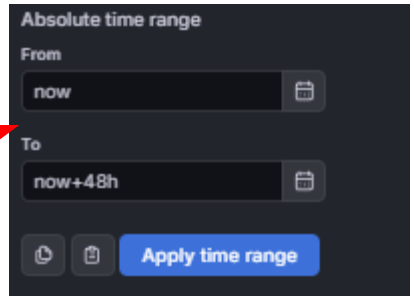
13. Zeitfilter einstellen (muss!)

→ From: now

→ to: now+48h (ohne Leerzeichen)

→ alle Daten sollten vorhanden sein





Anlage [redacted] | Alternative Aktionen EIN | PV-Fenster | Dashboard | Entscheidungs-DB | now to now+48h CEST | Refresh 30s

Mögliche Aktion

innerhalb des Prognosezeitraums

Solarthermie

neue mögliche Aktion voraussichtlich am 21.04.2026 nach 17:00 Uhr

[redacted]

8.9 °C

21.04.2026

11:55

Mögliche Maßnahmen (Wirkung von links nach rechts abnehmend)

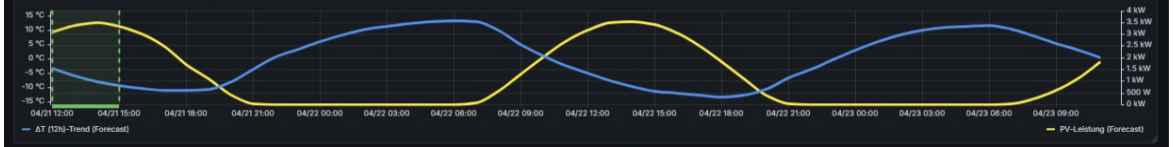
Wirkung	Offset der Heizkurve	Raum-Soll-Temperatur ändern	Nachtabsenkung	Zeitprogramme ändern
Heizbedarf kurzfristig erhöhen	+1 K	+0,5 bis +1 Grad	um 1 Grad erhöhen	Heizbeginn an PV anpassen

Aktions-Zeitplan

ΔT (12h) Prognose aktuell	Komfortzeit	nächstes PV-Fenster	PV-Leistung und Peak nächstes PV-Fenster			PV Leistung	Einstrahlung
-3.2 °C	6.1 Std.	21.04.2026 12:00	PV-Leistung 13.31 kW	Peak 3.51 kW	63%	gut nutzbar	684.0 W/m²

48-Stunden-Prognose der Energiemanagement-Parameter

Temperaturtrend-Entwicklung & PV-Leistung mit PV-Fenster



1 Heizen vorziehen

Parameter	Wert	Grenzwert
ΔT (12h) Prognose aktuell	-3.2 °C	fällt um mehr als 3 Grad
ergänzende Informationen:		
Komfortzeit	6.1	< 6 Stunden
Verdichter-LZ letzte Stunde	25 Minuten	weniger als 30 Minuten
Kompressorstarts letzte Stunde	0	<= 2
Warmwasser-Temp letzte Stunde	59.2 °C	<= 50 Grad

2 PV nutzen

Parameter	Wert	Grenzwert
ΔT (12h) Prognose aktuell	-3.2 °C	fällt um mehr als 3 Grad
nächster PV Peak	3.51 kW	>= 2.0 KW
ergänzende Informationen:		
Komfortzeit	6.1	
nächstes PV-Fenster	in ca. 0.0 Stunden	<= Komfortzeit
Verdichter-LZ letzte Stunde	25 Minuten	25 - 45 Minuten
Kompressorstarts letzte Stunde	0	<= 2
Warmwasser-Temp letzte Stunde	59.2 °C	> 50 Grad

3 Heizen verschieben

Parameter	Wert	Grenzwert
ΔT (12h) Prognose aktuell	-3.2 °C	> 3 Grad
ergänzende Informationen:		
Komfortzeit	6.1	> 6 Stunden
nächstes PV-Fenster	in ca. 0.0 Stunden	<= Komfortzeit
Kompressor-Laufzeit letzte Stunde	25 Minuten	< 20
Kompressorstarts letzte Stunde	0	<= 2
Warmwasser-Temp letzte Stunde	59.2 °C	> 50 Grad

4 Keine Änderung

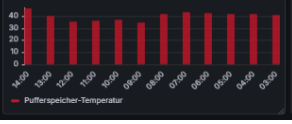
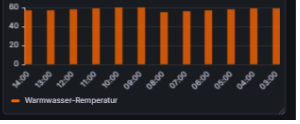
Parameter	Wert	Grenzwert
automatische Anlagen-Steuerung		
ergänzende Informationen:		
Komfortzeit	6.1	<= 6 Stunden
Verdichter-LZ letzte Stunde	25 Minuten	n/a
Kompressorstarts letzte Stunde	0	<= 2
Warmwasser-Temp letzte Stunde	59.2 °C	> 50 Grad

Temperaturverlauf & Komfortzeit-Verlauf mit PV-Fenster

Verdichterlaufzeit (min/h) ab jetzt

Kompressorstarts je Stunde ab jetzt

Pufferspeicher-Temperatur am Ende der Stunde



Warmwasser-Temperatur am Ende der Stunde